

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Параллельные алгоритмы»
Тема: Основы работы с потоками

Студент гр. 3343

Пименов П.В.

Преподаватель

Сергеева Е.И.

Санкт-Петербург

2025

Задание

Реализовать многопоточное умножение матриц с блочным разбиением.

1.1 Использовать `std::thread` для создания потоков. Провести исследование метрик производительности в зависимости от количества потоков и в зависимости от размера задачи, выбрать количество потоков при котором достигается оптимальная производительность.

1.2 Использовать `std::async` и `std::future`.

Реализация должна содержать тесты на корректность вычислений: состоящие в сравнение с однопоточной реализацией перемножения матриц. Метрики производительности приводить с осреднением, в отчете построить графики.

Выполнение работы

Все пункты задания успешно выполнены. В качестве `Matrix` используется `vector<vector<double>>`.

- `Matrix generateMatrix` – генерирует матрицу со случайными значениями, используя генератор `mt19937`
- `Matrix multiplyClassic` – перемножает две матрицы наивным методом
- `size_t optimalBlockSize` – определяет оптимальный размер блока матрицы для блочного умножения
- `void blockMultiply` – перемножает два блока матриц
- `Matrix multiplyThreaded` – блочное перемножение матриц с использованием `std::thread`
- `Matrix multiplyAsync` – блочное перемножение матриц с использованием `std::thread`
- `bool equal` – проверка корректности умножения

Компиляция (gcc) бинарника производится со следующими флагами:

- `-O3` – максимальная оптимизация.

- `-march=native` – использовать инструкции конкретного CPU, на котором компилируется программа. Менее кроссплатформенно, но более производительнее для данного CPU.
- `-pthread` – подключает поддержку posix-потоков.

Была протестирована производительность существующего решения. Сначала сравнивалась производительность 1000-0-Classic vs 1000-4-Threads vs 1000-4-Async (нотация в формате размер-потоки-метод). Результаты см. на Рисунке 1.

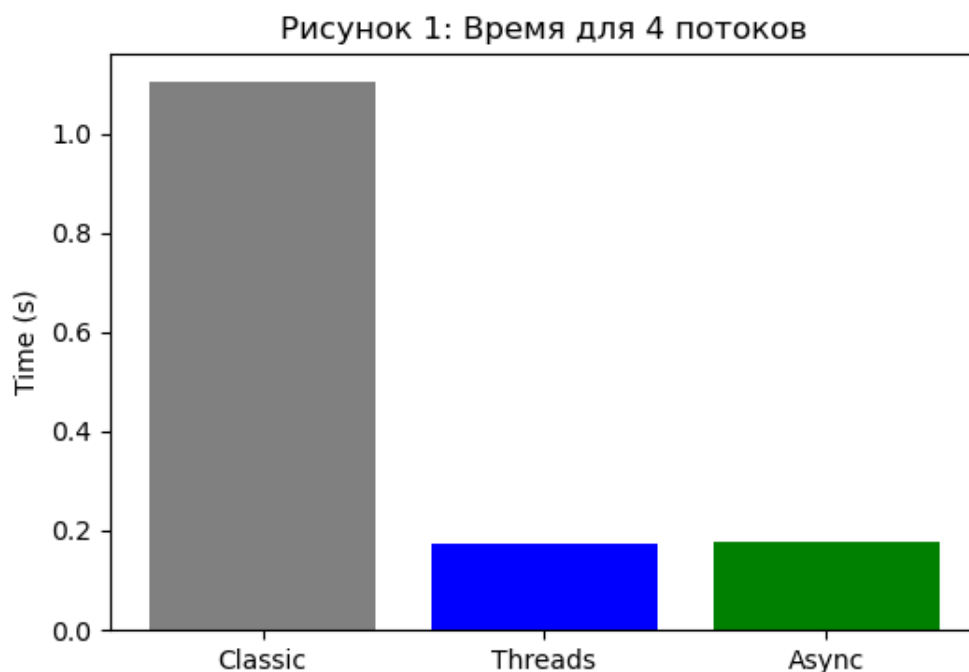


Рисунок 1 – Сравнение Classic vs Threads vs Async (1000-4(0))

Как видно по графику, классический (наивный) метод отстает от конкурентов примерно в 6 раз (в текущей конфигурации). Async и Threads на таком размере и количестве потоков показывают почти равное время выполнения.

Далее тестируется производительность Threads. Результаты см. на Рисунке 2.

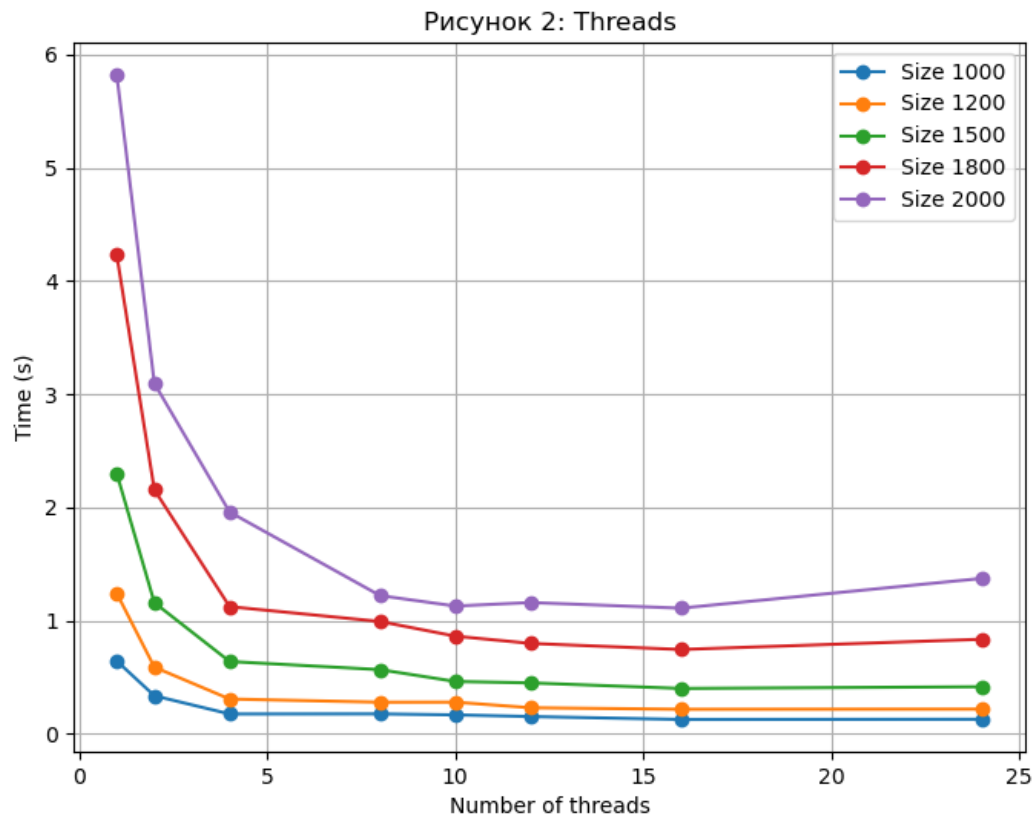


Рисунок 2 – Производительность метода Threads

Использованы размеры матриц: 1000-1200-1500-1800-2000. Использованы количества потоков: 1-2-4-8-10-12-16-24. Как видно по графику, наиболее оптимальное количество потоков для минимизации времени выполнения составляет 16 шт. Это имеет смысл, поскольку процессор, на котором проводились измерения имеет 8 ядер и может использовать по 2 потока на ядро (т.о. получая 16 логических процессоров). Больше число потоков показывает ухудшение производительности. Далее тестируется производительность метода Async. Результаты см. на Рисунке 3.

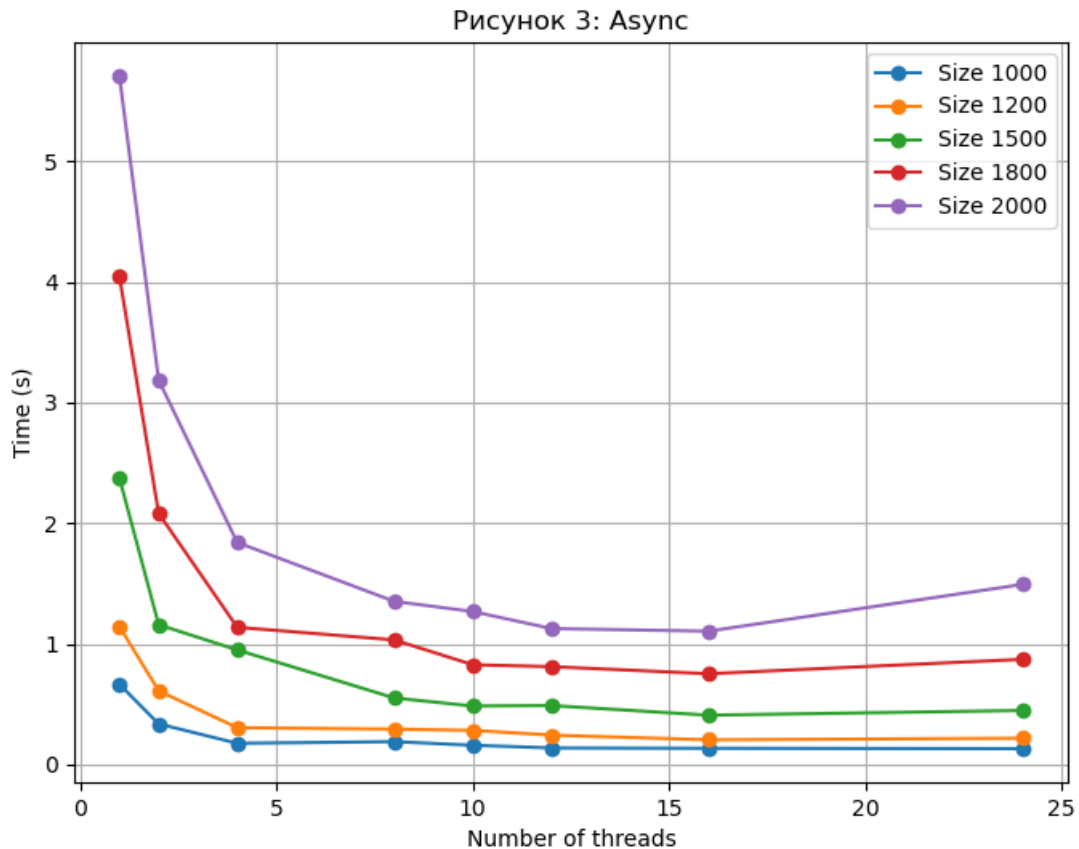


Рисунок 3 – Производительность метода Async

Результаты этого теста аналогичны прошлому. Async, так же как и Threads, наиболее эффективен на 16 потоках. Теперь посмотрим на ускорение методов Async и Threads относительно Classic. Результаты теста см. на Рисунке 4.

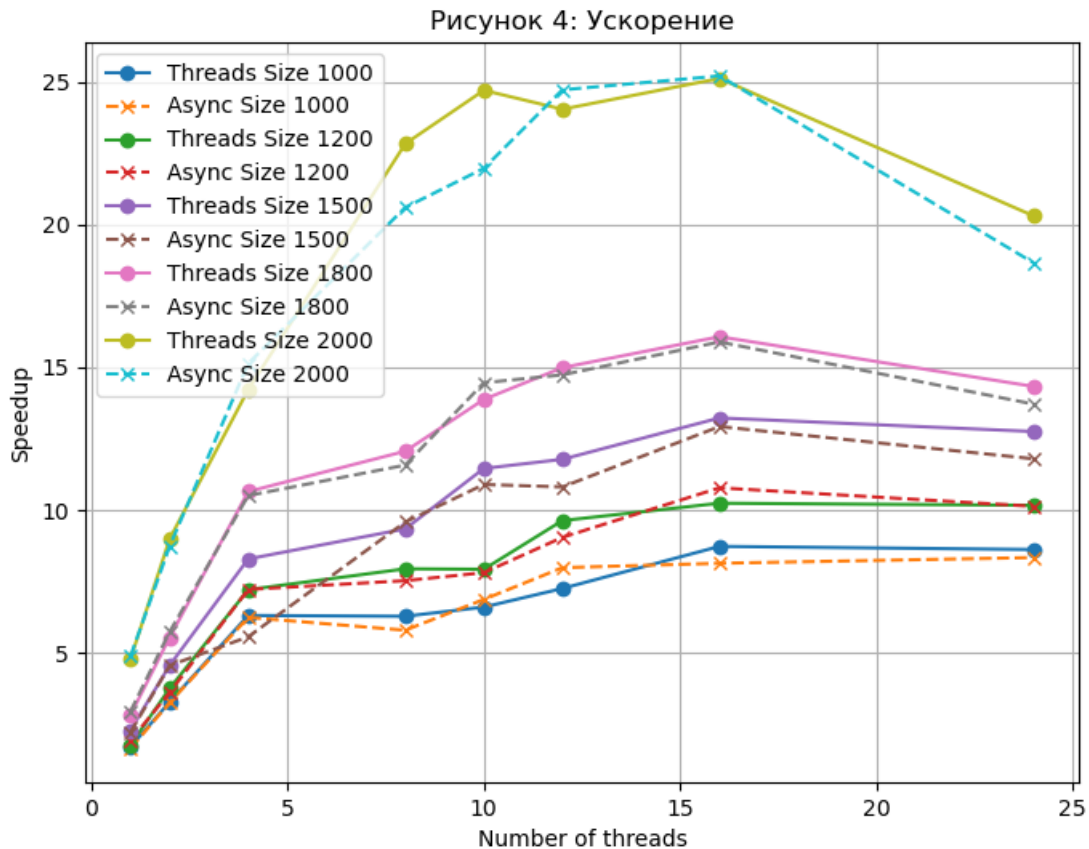


Рисунок 4 – Ускорение Async и Threads относительно Classic.

Чтобы корректно интерпретировать результаты по графику, нужно найти вертикальную, на которой точки графиков находятся выше, чем на остальных таких линиях. По графику видно, что на различных размерах матриц коэффициент ускорения достигает значений от 5 до 25, что показывает неэффективность классического (наивного метода). Также можно заметить, что Threads чуть производительнее Async (ломанные Threads более часто выше ломаных Async, значит, ускорение больше у Threads).

Исходный код программы и необработанные (raw) результаты тестирования можно увидеть в Приложении А.

Выводы

В ходе исследования были изучены основы работы с потоками, было реализовано многопоточное умножение матриц с блочным разбиением с использованием `std::thread`, `std::async` и `std::future`. Тестирование производительности показало:

- Классический метод умножения матриц сильно уступает в эффективности блочному умножению матриц с использованием `std::threads` или `std::async`
- Оптимальное количество потоков для умножения методами `Threads` и `Async` составляет 16 (что объясняется тем, что компьютер для тестов имеет 16 логических процессоров).
- Коэффициент ускорения `Threads` и `Async` относительно классического метода составляет от 5 до 25 при различных размерах матриц и количестве потоков.
- Метод `Threads` немного производительнее `Async` при прочих равных.

ПРИЛОЖЕНИЕ А

Файл main.cpp:

```
#include <algorithm>
#include <chrono>
#include <cmath>
#include <future>
#include <iostream>
#include <random>
#include <thread>
#include <vector>

using Matrix = std::vector<std::vector<double>>>;

// ===== УТИЛИТЫ =====
Matrix generateMatrix(size_t n, size_t m, int seed = 42) {
    std::mt19937 gen(seed);
    std::uniform_real_distribution<double> dist(0.0, 10.0);
    Matrix A(n, std::vector<double>(m));
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < m; ++j)
            A[i][j] = dist(gen);
    return A;
}

Matrix multiplyClassic(const Matrix& A, const Matrix& B) {
    size_t n = A.size(), m = B[0].size(), k = B.size();
    Matrix C(n, std::vector<double>(m, 0.0));
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < m; ++j)
            for (size_t t = 0; t < k; ++t)
                C[i][j] += A[i][t] * B[t][j];
    return C;
}

// ===== Автоматический выбор блока =====
size_t optimalBlockSize() {
    // приближённый размер кэша L1 (в байтах)
    const size_t L1_CACHE_SIZE = 32 * 1024;
    // double занимает 8 байт, три матрицы в блоке (A,B,C)
    size_t blockBytes = 3 * sizeof(double);
    size_t blockSize =
        static_cast<size_t>(std::sqrt(L1_CACHE_SIZE / blockBytes));
    return std::max<size_t>(8, blockSize); // минимум 8
}

// ===== Блочное умножение =====
void blockMultiply(const Matrix& A, const Matrix& B, Matrix& C,
    size_t blockSize, size_t rowStart, size_t rowEnd) {
    size_t n = A.size(), m = B[0].size(), k = B.size();
    for (size_t ii = rowStart; ii < rowEnd; ii += blockSize)
        for (size_t jj = 0; jj < m; jj += blockSize)
            for (size_t kk = 0; kk < k; kk += blockSize)
                for (size_t i = ii; i < std::min(ii + blockSize, rowEnd); ++i)
                    for (size_t j = jj; j < std::min(jj + blockSize, m); ++j)
                        for (size_t t = kk; t < std::min(kk + blockSize, k); ++t)
                            C[i][j] += A[i][t] * B[t][j];
}

// ===== std::thread =====
Matrix multiplyThreaded(const Matrix& A, const Matrix& B, size_t threads) {
    size_t n = A.size();
```



```

Matrix C(n, std::vector<double>(B[0].size(), 0.0));

size_t blockSize = optimalBlockSize();
size_t rowsPerThread = (n + threads - 1) / threads;
std::vector<std::thread> pool;

for (size_t t = 0; t < threads; ++t) {
    size_t start = t * rowsPerThread;
    size_t end = std::min(start + rowsPerThread, n);
    if (start >= n)
        break;
    pool.emplace_back(blockMultiply, std::cref(A), std::cref(B),
                      std::ref(C), blockSize, start, end);
}

for (auto& th : pool)
    th.join();
return C;
}

// ===== std::async =====
Matrix multiplyAsync(const Matrix& A, const Matrix& B, size_t threads) {
    size_t n = A.size();
    Matrix C(n, std::vector<double>(B[0].size(), 0.0));

    size_t blockSize = optimalBlockSize();
    size_t rowsPerThread = (n + threads - 1) / threads;
    std::vector<std::future<void>> futures;

    for (size_t t = 0; t < threads; ++t) {
        size_t start = t * rowsPerThread;
        size_t end = std::min(start + rowsPerThread, n);
        if (start >= n)
            break;
        futures.emplace_back(std::async(std::launch::async, blockMultiply,
                                       std::cref(A), std::cref(B), std::ref(C),
                                       blockSize, start, end));
    }

    for (auto& f : futures)
        f.get();
    return C;
}

// ===== Проверка и тестирование =====
bool equal(const Matrix& A, const Matrix& B, double eps = 1e-6) {
    if (A.size() != B.size() || A[0].size() != B[0].size())
        return false;
    for (size_t i = 0; i < A.size(); ++i)
        for (size_t j = 0; j < A[0].size(); ++j)
            if (std::abs(A[i][j] - B[i][j]) > eps)
                return false;
    return true;
}

// ===== main =====
int main() {
    std::vector<size_t> sizes = {1000, 1200, 1500, 1800,
                                2000}; // размеры матриц
    std::vector<size_t> thread_counts = {1, 2, 4, 8,
                                         10, 12, 16, 24}; // число потоков

    std::cout << "size,threads,method,time_s\n";
    for (size_t n : sizes) {

```

```

Matrix A = generateMatrix(n, n, 1);
Matrix B = generateMatrix(n, n, 2);

// Classic
auto start = std::chrono::high_resolution_clock::now();
Matrix C_ref = multiplyClassic(A, B);
auto end = std::chrono::high_resolution_clock::now();
double t_classic = std::chrono::duration<double>(end - start).count();
std::cout << n << ",0,Classic," << t_classic
          << "\n"; // 0 потоков для classic

for (size_t th : thread_counts) {
    // Threads
    start = std::chrono::high_resolution_clock::now();
    Matrix C_thr = multiplyThreaded(A, B, th);
    end = std::chrono::high_resolution_clock::now();
    double t_threads =
        std::chrono::duration<double>(end - start).count();
    std::cout << n << "," << th << ",Threads," << t_threads << "\n";

    // Async
    start = std::chrono::high_resolution_clock::now();
    Matrix C_async = multiplyAsync(A, B, th);
    end = std::chrono::high_resolution_clock::now();
    double t_async = std::chrono::duration<double>(end - start).count();
    std::cout << n << "," << th << ",Async," << t_async << "\n";

    // Проверка корректности (опционально)
    if (!equal(C_ref, C_thr))
        std::cerr << "Threaded FAIL size=" << n << " th=" << th << "\n";
    if (!equal(C_ref, C_async))
        std::cerr << "Async FAIL size=" << n << " th=" << th << "\n";
}
}
}

```

Необработанные результаты тестирования производительности:

```

size,threads,method,time_s
1000,0,Classic,1.10581
1000,1,Threads,0.643192
1000,1,Async,0.666201
1000,2,Threads,0.335759
1000,2,Async,0.337751
1000,4,Threads,0.174975
1000,4,Async,0.176862
1000,8,Threads,0.175727
1000,8,Async,0.190538
1000,10,Threads,0.167216
1000,10,Async,0.160695
1000,12,Threads,0.152235
1000,12,Async,0.138351
1000,16,Threads,0.126551
1000,16,Async,0.135745
1000,24,Threads,0.128174
1000,24,Async,0.132494
1200,0,Classic,2.21797
1200,1,Threads,1.24112
1200,1,Async,1.14511
1200,2,Threads,0.588672
1200,2,Async,0.611163
1200,4,Threads,0.307069
1200,4,Async,0.306704
1200,8,Threads,0.278894

```

1200,8,Async,0.294281
1200,10,Threads,0.279429
1200,10,Async,0.283708
1200,12,Threads,0.230093
1200,12,Async,0.245009
1200,16,Threads,0.21643
1200,16,Async,0.205627
1200,24,Threads,0.217981
1200,24,Async,0.218756
1500,0,Classic,5.29816
1500,1,Threads,2.30011
1500,1,Async,2.37601
1500,2,Threads,1.15824
1500,2,Async,1.15963
1500,4,Threads,0.638537
1500,4,Async,0.950465
1500,8,Threads,0.566631
1500,8,Async,0.552337
1500,10,Threads,0.46214
1500,10,Async,0.485925
1500,12,Threads,0.449621
1500,12,Async,0.489784
1500,16,Threads,0.400386
1500,16,Async,0.409747
1500,24,Threads,0.415325
1500,24,Async,0.44894
1800,0,Classic,11.9682
1800,1,Threads,4.22948
1800,1,Async,4.04788
1800,2,Threads,2.15489
1800,2,Async,2.08141
1800,4,Threads,1.12315
1800,4,Async,1.1382
1800,8,Threads,0.991746
1800,8,Async,1.0334
1800,10,Threads,0.86233
1800,10,Async,0.827506
1800,12,Threads,0.798338
1800,12,Async,0.812039
1800,16,Threads,0.744511
1800,16,Async,0.753119
1800,24,Threads,0.834994
1800,24,Async,0.872636
2000,0,Classic,27.8632
2000,1,Threads,5.82146
2000,1,Async,5.70595
2000,2,Threads,3.09506
2000,2,Async,3.18522
2000,4,Threads,1.95886
2000,4,Async,1.84102
2000,8,Threads,1.22063
2000,8,Async,1.35269
2000,10,Threads,1.12827
2000,10,Async,1.26858
2000,12,Threads,1.15935
2000,12,Async,1.12748
2000,16,Threads,1.10964
2000,16,Async,1.10577
2000,24,Threads,1.37257
2000,24,Async,1.49333